HW4 Report

Jinxi Xiao
2021533005
ShanghaiTech University

Abstract

This report contains my solutions to the given problems as well as some of my own understandings. In addition, the sections listed in this report does not one-to-one correspond to the problems, but they do overall cover the whole problem set. [1]

1. Deriving the Jacobian Matrix

It should be aware that the following proof will ignore the conversion between Euclidean form and the Homogeneous form of the coordinate. Since the process will be simpler and this is automatically achieved when using some cpp libraries like EIGEN.

We left-multiply a small perturbation $\Delta\boldsymbol{\xi}$ to the calculated transformation $\exp(\boldsymbol{\xi}^\wedge)$, then we get

$$\begin{aligned}
&e\left(\boldsymbol{\xi} \oplus \Delta\boldsymbol{\xi}\right) \\
&= \boldsymbol{I}_1\left(\frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P}\right) - \boldsymbol{I}_2\left(\frac{1}{Z_2}\boldsymbol{K}\exp\left(\Delta\boldsymbol{\xi}^\wedge\right)\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P}\right) \\
&\approx \boldsymbol{I}_1\left(\frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P}\right) - \boldsymbol{I}_2\left(\frac{1}{Z_2}\boldsymbol{K}\left(1+\Delta\boldsymbol{\xi}^\wedge\right)\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P}\right) \\
&= \boldsymbol{I}_1\left(\frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P}\right) - \\
&\boldsymbol{I}_2\left(\frac{1}{Z_2}\boldsymbol{K}\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P} + \frac{1}{Z_2}\boldsymbol{K}\Delta\boldsymbol{\xi}^\wedge\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P}\right)
\end{aligned} \tag{1}$$

define

$$\boldsymbol{q} = \Delta\boldsymbol{\xi}^\wedge\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{P}, \quad \boldsymbol{u} = \frac{1}{Z_2}\boldsymbol{K}\boldsymbol{q}$$

where $\boldsymbol{q}$ is the coordinate of the $\boldsymbol{P}$ after perturbation, and $\boldsymbol{u}$ is the corresponding pixel coordinate.

Using Taylor expansion, we can obtain

$$\begin{aligned}
&e\left(\boldsymbol{\xi} \oplus \Delta\boldsymbol{\xi}\right) \\
&= \boldsymbol{I}_1\left(\frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P}\right) - \boldsymbol{I}_2\left(\frac{1}{Z_2}\boldsymbol{K}\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P} + \boldsymbol{u}\right) \\
&\approx \boldsymbol{I}_1\left(\frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P}\right) - \boldsymbol{I}_2\left(\frac{1}{Z_2}\boldsymbol{K}\exp\left(\boldsymbol{\xi}^\wedge\right)\boldsymbol{P}\right) - \frac{\partial\boldsymbol{I}_2}{\partial\boldsymbol{\Delta\xi}}\Delta\boldsymbol{\xi} \\
&= e\left(\boldsymbol{\xi}\right) - \frac{\partial\boldsymbol{I}_2}{\partial\boldsymbol{u}}\frac{\partial\boldsymbol{u}}{\partial\Delta\boldsymbol{\xi}}\Delta\boldsymbol{\xi}
\end{aligned} \tag{2}$$

As a result, we can conclude that the Jacobian would be

$$\boldsymbol{J} = -\frac{\partial\boldsymbol{I}_2}{\partial\boldsymbol{u}}\frac{\partial\boldsymbol{u}}{\partial\Delta\boldsymbol{\xi}}\Delta\boldsymbol{\xi} \tag{3}$$

where

$$\frac{\partial\boldsymbol{u}}{\partial\Delta\boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_xX}{Z^2} & -\frac{f_xXY}{Z^2} & f_x + \frac{f_xX^2}{Z^2} & -\frac{f_xY}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_yY}{Z^2} & -f_y - \frac{f_yY^2}{Z^2} & \frac{f_yXY}{Z^2} & \frac{f_yX}{Z} \end{bmatrix}$$

and

$$\boldsymbol{q} = [X, Y, Z]^T$$

2. Direct Odometry

2.1. Alignment of timestamps

In order to make the following work efficient, I have aligned the timestamps between **rgb/** and **depth/** into pairs. The maximum gap of time is 0.015(s), and there are 754 pairs in total.

2.2. Dense Tracking

I adopt the idea of pyramid whole image alignment in DTAM [1]. That is, we will align two consecutive images at a higher level(low resolution) first and then to the lower level. This strategy helps to make the optimization converge.

In detail, all pixels that are at least 20 pixels away from the edge of the image will be chosen as "features" needed

[1] Codes are stored at my github repo

to pair. And to make it more robust, I further select pixels whose norm of gradient greater than 50(However, there are still around 20k pixels involved in optimization process). In addition, for pixels whose error(projected gray image value difference) are greater than 25, they will be exclude from the optimization process.

Some numerical results can be found in Table 1. But there's one thing that I would like to mention. It is that to make the whole process faster, I adopt the method of parallel programming, as a result, the order(sequence) of how each pixels get optimized in each running process is distinct. Therefore, we may observe a total different performance when we run the program for several times.

2.3. Sparse Tracking

In sparse tracking, only pixels with norm of gradient greater than 120 are chosen. And I have implemented pattern 1 and pattern 8 in 1 introduced in DSO [2]. More results can be found in the later section 2.4.

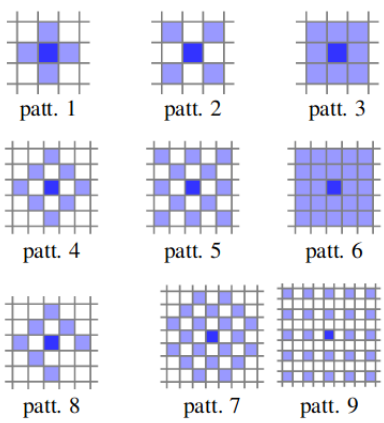

Figure 1. Pattern used in DSO

One thing that is interesting is that the performance of pattern 8 is worse than pattern 1, which is not consistent to the results in the DSO paper. Personally, I believe the reason is that they have implemented more than I do, and my algorithm is much simpler. Or we can say that the whole tracking process is distinct.

2.4. Results

Some visual results are shown in Figure 2 and Figure 3. Here are some numerical results.

	Time(s)	RMSE	SSE
Dense	14.6	0.2608	51.14
Sparse_patt1	13.0	0.2258	38.34
sparse_patt8	14.3	0.2452	45.21

Table 1. The time and error of three tracking algorithms.

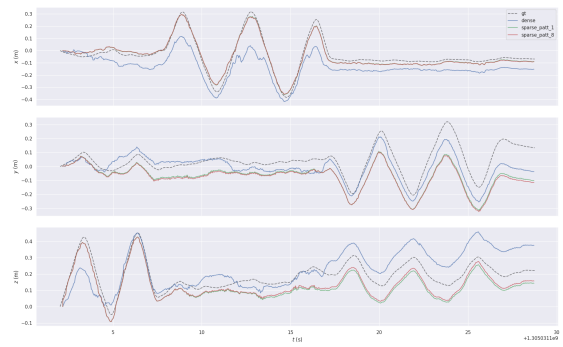

Figure 2. XYZ performance of the three algorithms

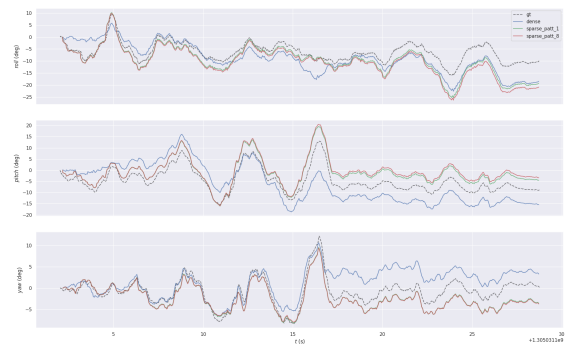

Figure 3. RPY(Roll Pitch Yaw) performance of the three algorithms

References

[1] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "Dtam: Dense tracking and mapping in real-time," in *2011 International Conference on Computer Vision*, pp. 2320–2327, 2011.

[2] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, p. 611–625, Mar. 2018.